

How to Build a Research Platform for Browser Exploit Development

Introduction.....	2
General Instructions.....	2
Platform OS.....	2
Virtual Machine Platform.....	3
Build Options & Configurations.....	3
Build Targets & Usage.....	4
Hardware Requirements.....	5
Storage.....	5
CPU & RAM.....	6
VM Instructions.....	6
Setup & Import.....	7
Locations & Scripts.....	7
Checkout & Branches.....	8
Building & Targets.....	9
Debugging & Introspection.....	9
DIY Build.....	10
Mozilla Firefox.....	10
Google Chromium.....	12
Setup.....	12
Build.....	15
Troubleshooting.....	16
Safari & WebKit.....	17
References.....	18
Troubleshooting.....	19
OOM Workarounds.....	19
Package Dependency Issues.....	20
Project Dependency Issues.....	21
VirtualBox x Ubuntu Issues.....	22

Introduction

Browser builds that are publicly available from app stores and vendor websites are not suitable for exploit development research. You will need a locally hosted debug build of your target.

Building modern browsers and JavaScript engines is not hard, as long as you follow a small set of “know-how” guidelines that will prevent you from applying the software vendor’s official building instructions in the wrong context, which is a common reason for getting build errors and frustration with mature codebases. This document outlines such guidelines together with just-tested build algorithms for each major browser, as well as common issues with the build process and the author’s personal tips. It also contains specific instructions on how to work with the exercise VM which comes with Zero Day Engineering [Browser Exploit Design training](#).

Computation resources are important for the build process, as browsers are large software systems. Specific requirements will be different for different build types and targets, which allows you to build at least “something” usable even on a limited hardware. On the other hand, certain build configurations will require an upgraded machine, such as a full CodeQL build of Chromium browser. More on that in the next sections.

If you are short on time, skip forward to the [VM Instructions](#) section. The virtual machine contains code checkout of all major browsers, as well as pre-built binaries targeting our training exercises.

General Instructions

Platform OS

We will use modern Linux for building everything (Ubuntu 24.04.01 LTS). This services several goals.

Linux is ideal as the primary platform for low-level security research because of the openness of the system environment to configuration, hacks and privileged access. In addition, it has a well-developed and consistently updated ecosystem of debugging and introspection tools that make low-level research convenient. If something isn’t working, you can always patch the OS code and rebuild it - not so much with more closed operating systems, where you are at the mercy of whatever “officially available” features and under the constraints of vendor’s ideas of usability and convenience because reverse-engineering and binary patching will cost you. Then, transferring the work from Linux to the “production target” OS is straightforward.

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

Browser exploit development knowledge is **largely OS-neutral**. If your goal is to develop exploits on Windows platform, you'll face substantial differences from Linux only at the late stages of exploit development process, such as with heap grooming (which may depend on the OS allocator specifics), shellcode integration (that will use OS-specific API endpoints), and so on. Browsers have slightly different implementation and system internals on different OSes, but in most cases this differential will reside several layers deeper in the code stack below the vulnerability, and so we don't have to deal with it. An exception is Browser Sandbox, which uses OS-specific mechanisms for process isolation and enforcing privileges, so this part of research is somewhat less OS-neutral (but still not fully OS-specific).

In general, the key to smooth build is using a fresh install of the specific Linux version which is supported as per the project development setup instructions. Using your go-to Linux laptop isn't a good idea because package dependency issues can ruin even a well-tested mainstream software build, if you don't have experience with it. For browsers we use a modern version of Ubuntu Linux, but this is not a general rule. For example, certain less modern codebases will require an older version of Linux, and upgrading it will cost you.

Furthermore, we'll use LTS release (Long Term Service) because that's what mainstream software project developers use, for a variety of good reasons.

Virtual Machine Platform

A dedicated "bare-metal" setup with modern Linux is ideal as a build platform for all the browsers. I do not recommend using virtualization or containers for the build environment. But, this document is part of a VM-based redistributable setup for the training so, a few words about virtualization.

You can use any modern hypervisor that is able to virtualize Linux to create the virtual machine to build browsers. Browsers are not peculiar in that regard - as opposed to kernels or other hypervisors, where VM setup is sensitive. I recommend VirtualBox that can be installed from the package server or downloaded from the Oracle website. VMWare Workstation is another solid option.

Do not use shared folders to store the code. Shared folders use a custom protocol over the hypervisor interface to communicate file data, which is slow and potentially restricts some specialized filesystem features - that may lead to esoteric issues when running the build from a shared folder. Storing the codebases on the local drive in-vm would be the best option. Alternatively, connecting an external drive via USB pass-through can also work.

Build Options & Configurations

Browser codebases expose a variety of default build configurations, and further allow to fine-tune the configuration with many advanced options. Different build configs are ideal in

different stages of the R&D workflow. Normally you'll use several different builds of the same code version to work through a vulnerability exploit.

At the simplest level, historical convention gives us two general options for a project build: Release and Debug. Release builds will be smaller and faster, and it will be equivalent or very close at implementation level to the official public build that normal users have. However, it may be not identical - for example, Chromium codebase release build is not 100% same as the publicly available Chrome browser, hence the naming difference.

Debug build will include additional debug logging logic and CHECK's. It also will include some symbolic information, and more often than not it will be optimized for low-level debugging (if not, then you'll get the "optimized out" message when trying to access a local variable in debugger, despite using a "debug" build!). Specific configuration of Debug and Release builds is implementation-specific and varies from developer to developer.

In addition to these basics, various in-between build configurations may be available, optimized for different scenarios. For example, there can be a Debug build and a separate Debug-opt build, the latter being normally optimized (meaning, not good for actual debugging). Another example is fuzzing builds that require a specialized configuration cross between Release and Debug plus extra options added, such as sanitizer callbacks. More on that in the training.

Build Targets & Usage

In the exploit development workflow we use different builds for different purposes. **Release builds** are ideal for late stages of exploit development workflow where we work closely with the process memory layout. This is because a Debug build will mess up the allocator determinism with extra logging and other differences of the program algorithm, a sensitive issue for many memory corruption exploits, which is why you should never write the final exploit on the debug build.

On the other hand, **Debug builds** are ideal for bug triage and early stages of exploit development: symbolic information that they expose is essential to understand the bug; extra logging may help to understand the root cause; low optimization levels allow the use of advanced source-level techniques in CPU-level debugging - neither is granted in a Release build. A Release build may even fail to show the crash with a known-good vulnerability test case, which is normal.

Another aspect of debug configuration is the *build target*. By default, a typical build config will compile not just the browser but also all of its unit tests, fuzzers, tools and documentations - that aren't immediately useful for our purposes while taking away build time and file storage. For that reason it's a good idea to invest some time early on to keep your build lean.

JavaScript engine will be exposed as a separate build target. Meaning, it can be built in two ways: by building the whole browser - it will obviously have the engine inside, and usually, the repl shell binary will be produced as well - or, by building the engine stand-alone. The latter option is invaluable if you only want to attack the JSE, it builds fast and can be checked out outside of the browser tree as well. Personally I like to keep both a full browser build and a standalone javascript engine build in separate directories: so for example, if I want to test an older javascript engine vulnerability, I can checkout the code tree and build the standalone repo without breaking the full browser build with a potentially incompatible version of the javascript engine.

By default, the build script will usually build all the targets as they pertain to a typical project developer, including various unit tests and utilities, and you'd have to look under the hood of the build script to see the target list and strip it down to essentials. This step isn't mandatory as long as you have a big file storage, many cores and a lot of random access memory.

Hardware Requirements

A full browser codebase takes time to build. The time estimations given in official project build instructions tend to be incorrect, and so do the suggested hardware requirements. Importantly, build time depends on a variety of factors that come down to hardware resources and build configuration. This section deals with the former in a systematic way that is implementation-neutral and timeproof.

Storage

One variable in the hardware configuration is **file storage**. In addition to the codebases, build products take a *huge* amount of space - especially in Debug builds. Ideal solution is to get at least a 1+TB internal storage upgrade and forget about this problem for a couple of years. Internal SSD storage is best because disk access speed will affect building speed, so it's not a good idea to use an old external USB1.0 drive to hold the codebase. Partitioning the disk with a Linux-native filesystem format is pretty much a requirement: not just due to speed, but also, because many codebases have case-sensitive filenames that will suddenly start haunting you when the respective file gets an upstream update.

If you must optimize for disk space, then:

1. Strip down build targets to build only what you need. Getting started with just a standalone JavaScript engine build is a good idea.
2. Review Debug Symbols configuration in your build. This can be tweaked to include a minimal subset of symbols such as procedure names (useful for crash logs), but no full symbolic information, that will take less space.
3. Create a custom build script based on the default Release configuration, but with low code optimization (-O0, -Og) and added minimal symbolic information. This will save on

debugging logic. Some projects offer explicit build options to optimize storage at the expense of build time or debugging features availability.

Aside from storing the codebase and build products, storage can be used to enlarge the swap file, which may help to offset memory requirements, discussed below.

CPU & RAM

Build speed depends on the number of **CPU cores** and the amount of **RAM**. These two variables are tied together, because the more cores a build uses, the more memory it will take. Number of CPU cores is important because modern build systems parallelize, so the more cores, the faster the build; as long as they get enough RAM. A good rule of thumb is 4GB of RAM per core. In theory, you can also look into performance of various RAM technologies as well as CPU clock speed to fine-tune hardware upgrade choices, both would be vanishing in comparison to the overall computing capacity (number of cpu cores + volume of memory) and likely not worth the effort.

Certain build configurations are more sensitive to memory requirements than the others. A Debug build will typically need more RAM, because the compiler needs to hold a vast amount of string data in memory while it's linking large binaries. For a similar reason, CodeQL builds take enormous amounts of memory and are not able to continue after being killed by the OOM guard. As a general rule, the moment you see OOM kick during build means time to upgrade your hardware.

The system swap file helps to reduce memory stress and even to “extend memory” to some extent. Increase the swap file size to 4GB or ¼-th of the size of total RAM, whichever is larger. This can be finetuned by watching resource usage during build, you'll notice when and if the swap file reaches peak capacity and can use some extension.

Modern build systems are smart and iterative, recompiling and re-linking only those files that were modified against the last build. It means that you'll only need to run the full build *once* for the given version/commit. After that any minor code modifications will be compiled quickly, unless the “minor modification” affects some core program logic that changes many object modules up the call stack.

VM Instructions

Training VM is provided as a VirtualBox VDI file, which is an image of the HDD that you import into a new Virtual Machine configuration and boot from.

VM contains a fully set-up and tested building & debugging environment for Firefox, Chromium and Webkit, as of date in the footer. The OS is Ubuntu Desktop 24.04.1 LTS 64-bit.

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

Setup & Import

Install VirtualBox on any platform with x86_64 architecture. You can download the package from the app manager or from the [virtualbox.org](https://www.virtualbox.org) website, or build it yourself.

In VirtualBox, click the button “**Create** a new VM” and choose “Ubuntu (64-bit)”. On the next step of the setup menu, which is titled “Virtual Hard Disk”, instead of the default option “Create a Virtual Hard Disk Now”, select the option “Use an Existing Virtual Hard Disk File”. Specify the location of our VM VDI file and complete through the setup menu. After that the VM should boot into the Browser Exploitation Linux environment normally.

Note that VDI location will be difficult to change after this procedure, so put the virtual disk file in the final location before importing it.

For virtual hardware settings, use 8GB+ RAM and 2 CPUs. The more, the better, but stay in the green zone.

Access credentials, user/sudo: zerodaytraining, password: isawesome.

Locations & Scripts

Code is in ~/code.

Directory structure:

- code/mozilla-unified/: Firefox, SpiderMonkey and Thunderbird.
- code/chromium/: Chromium project
- code/depot_tools/: Google’s code management toolchain

Mozilla:

- mozconfig-*: our custom build configuration files
- obj-*: build products for different configurations
- obj-*/dist/bin/: binaries

Chrome:

- out/*: build products and binaries
- out/<buildname>/args.gn: build configuration file

Note that object files were removed from pre-built chrome directories to reduce the size of the VM. Meaning, you can still use the binaries, but incremental build won’t be available. Full build products can be recovered by simply running `autoninja -C out/<buildname> chrome` from our chrome build environment.

Checkout & Branches

Current version of code and build products on VM reflect exercise requirements for Browser Exploit Design training exercises:

Browser	Commit / tag	CVE	Notes
Google Chrome	84.0.4147.68	n/a, 2020	Sandbox escape: Mojo Use-after-free in RawClipboard implementation
Mozilla Firefox	ddac794b	CVE-2024-9680	Remote: DOM Use-after-free in Animation timelines
WebKit	538c84a3	CVE-2023-41993	Remote: JavaScriptCore optimization logic issue in DFG clobberize()

If you want to check out and build recent code, be aware of the consequences:

- Project dependencies may need a massive update.
- The build may overwrite current build products by default.
- You must allocate **a lot** of virtual RAM to the virtual machine before attempting build, otherwise it will crash from OOM.
- Review [DIY Build](#), [OOM Workarounds](#) and [Hardware Requirements](#) sections for detailed instructions on build setup.

The following instructions are to update project code from the upstream.

Mozilla:

```
$ cd mozilla-unified
$ git pull
```

Note, these instructions will checkout the latest commits on the development tree aka 'trunk', which isn't the same as the latest release code. To build an exact replica of a public release version, you'll need to find out the specific commit that it was built from, and check it out. Alternatively, use 'git checkout <tag>' if you know the version tag.

Chrome:

```
$ git rebase-update
$ gclient sync
```

To check out an older commit:

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025


```
$ git checkout <commit>
$ gclient sync -D
```

If the commit is very old, you may need to check out an older version of depot_tools as well by matching the date.

WebKit:

```
$ git checkout
```

Building & Targets

Firefox SpiderMonkey engine shell only, debug profile with disabled optimization:

```
env MOZCONFIG=mozconfig-js-debug ./mach build
// build products in obj-js-debug
// timing: ~10 minutes
```

Firefox SpiderMonkey engine shell only, release profile with optimized code:

```
env MOZCONFIG=mozconfig-opt-js ./mach build
// build products in obj-js-opt
// timing: ~10 minutes
```

Firefox browser build from current code tree, standard release-development profile (Nightly build):

```
./mach build
// build products in obj-x86_64-pc-linux-gnu, default setup
// timing: ~1 hour
```

Chromium default development build (“Release + Asserts”):

```
autoninja -C out/Default
// build products in out/Default directory
// timing: 4+ hours
```

Chromium non-debug build with AddressSanitizer (not suitable for debugging):

```
$ autoninja -C out/asan
// build products in out/asan directory
```

Debugging & Introspection

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

Enable debugging system-wide:

```
root# echo 0 > /proc/sys/kernel/yama/ptrace_scope
```

Start Firefox browser with gdb attached to the browser (parent) process:

```
./mach run --debug
```

Attach gdb to Firefox renderer process:

```
(gdb) attach <pid> // hover tab to see pid
```

Attach gdb to WebKitGTK MiniBrowser renderer process:

```
Tools/Scripts/run-minibrowser --gtk --debug // start fresh
ps ax | grep WebKitWebProcess // 2nd process is the currently open tab
sudo gdb --attach <pid>
(gdb) set substitute-path /app/webkit /home/zerodaytraining/code/WebKit
```

DIY Build

This section outlines implementation-specific instructions to build a vulnerability research platform for each major browser from scratch, without using the training VM. Instructions were tested on {see footer for the last-modified timestamp}, and are guaranteed to work if followed precisely.

Instructions assume a fresh installation of Ubuntu Desktop 24.04.01 LTS 64-bit (x86_64) as the operating system.

Create a directory that will hold all the code and cd into it. I'll be using ~/code.

Mozilla Firefox

Install dependencies:

```
$ sudo apt update && sudo apt install python3 git curl python3-pip
python3-mercurial
```

Test mercurial, the Mozilla project's historical code management system:

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

```
$ hg version
```

You may need to install git-cinnabar manually (mercurial to git frontend):

```
$ curl -O
https://raw.githubusercontent.com/glandium/git-cinnabar/master/download.py
$ ./download.py
$ mv git-cinnabar git-remote-hg /usr/local/bin
```

Test git-cinnabar:

```
$ git-remote-hg
```

Next, download & run the codebase bootstrapping script:

```
$ curl
https://hg.mozilla.org/mozilla-central/raw-file/default/python/mozboot/bin/bootstrap.py -O

$ python3 bootstrap.py --vcs=git
```

This will check out mozilla project code into mozilla-unified directory, and configure it to use git for version control. Code download takes about 30 minutes on a 50 Mbps internet connection.

Click through the post-download setup by choosing build target to be “2. Firefox for Desktop”. This will also install system dependencies.

Checkout the latest code and tags from the upstream source and update the working directory:

```
$ git-cinnabar fetch --tags
$ git pull
```

Check that the code is up to date by running git log and notice the date/time of last commit.

Note: bookmarks/central (the default branch that git will pull) is the "master" branch of Firefox source code. Other available bookmarks are listed at <https://hg.mozilla.org/mozilla-unified/bookmarks>.

Build Firefox:

```
$ ./mach build
```

Mozilla build system uses “mozbuild” configuration files under the hood to set up the build. Above steps will use the default config which builds the whole browser from the trunk in

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

standard release profile (aka, “Nightly build”). You can create separate mozbuild files for different targets and configs, and switch-build between them like this:

```
$ env MOZCONFIG=/path/to/mozconfig ./mach build
-or-
$ export MOZCONFIG=/path/to/mozconfig && ./mach build
```

In this way you can tell the mach build system to build, for example, only the standalone javascript engine with a shell binary, or a special build of the browser optimized for fuzzing.

There are a bunch of mozconfig files in the “build” subdirectory, which you can use for inspiration to tweak your build configs. Also, refer to the “Configuring Build Options” link in the References section.

Clean build (purge object files & compile from scratch):

```
$ ./mach clobber && ./mach build
```

See `./mach --help` for commands.

Binaries: in `obj-x86_64-pc-linux-gnu/dist/bin` (default configuration). You can also use `./mach run` to launch the current build binary.

Google Chromium

Chromium build system doesn’t support Ubuntu 24.04 at the moment, so we’ll build Chromium in a Docker container while Google is updating their infra.

The following instructions can be split in three phases:

1. Checkout chromium code on the host.
2. Install and set up the Docker rootless environment.
3. Build Chromium inside the container.

Note: Docker build setup is extremely messy and unstable; if you are building only Chrome for yourself, consider using a native installation of Ubuntu 22.04.1 (or whatever is officially supported by Google at the moment) as the build platform.

Setup

On the host, pre-install basic dependencies:

```
$ sudo apt update && sudo apt install python3 git
```

Install `depot_tools`, which is Google’s utility toolchain to manage the codebase:

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

```
(in ~/code)
$ git clone
https://chromium.googlesource.com/chromium/tools/depot\_tools.git
$ export PATH="${HOME}/code/depot_tools:$PATH"
$ echo `export PATH="${HOME}/code/depot_tools:$PATH"` >> ~/.bashrc
```

Checkout the code using fetch command from depot_tools:

```
$ mkdir chromium && cd chromium
$ fetch --nohooks chromium
```

This command will take 2++ hours on our setup with 100Mbps internet connection.

Next, install docker in rootless mode. There are different ways to do it, I used this algorithm:

1. Install Docker from Ubuntu Software app.
2. Follow vendor's instructions to make it rootless:
<https://docs.docker.com/engine/security/rootless/>.

You can skip pt.2 and let docker run as root, not recommended.

Ubuntu 24 [restricts unprivileged user namespaces](#), which are required for rootless mode to work, as well as for chrome dev build sandbox, so we need to disable this setting:

```
root# echo kernel.apparmor_restrict_unprivileged_userns=0 | sudo tee
/etc/sysctl.d/60-apparmor-namespaces.conf && reboot
-or- (fix until next reboot)
root# sysctl -w kernel.apparmor_restrict_unprivileged_unconfined=0
root# sysctl -w kernel.apparmor_restrict_unprivileged_userns=0
```

After Docker installation is ready, create a Dockerfile with these contents:

```
# Use an official Ubuntu base image with Docker already installed
FROM ubuntu:22.04

# Set environment variables
ENV DEBIAN_FRONTEND=noninteractive

# Install Mantatory tools (curl git python3) and optional tools (vim sudo)
RUN apt-get update && \
    apt-get install -y curl git lsb-release python3 git file vim sudo && \
    rm -rf /var/lib/apt/lists/*

# Export depot_tools path
ENV PATH="/depot_tools:${PATH}"
```

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

```

# Configure git for safe.directory
RUN git config --global --add safe.directory /depot_tools && \
    git config --global --add safe.directory /chromium/src

# Set the working directory to the existing Chromium source directory.
# This can be either "/chromium/src" or "/chromium".
WORKDIR /chromium/src

# Expose any necessary ports (if needed)
# EXPOSE 8080

# Create a dummy user and group to avoid permission issues
RUN groupadd -g 1000 builder && \
    useradd -u 1000 -g 1000 -m builder

# Create normal user with name "builder". Optional and you can use root but
# not advised.
USER builder

# Start Chromium Builder "chrom-d" (modify this command as needed)
# CMD ["autoninja -C out/Default chrome"]
CMD ["bash"]

```

The script works by creating a Ubuntu 22.04 environment and exposing the host's chromium code directory to it, plus some extra setup.

Build the container:

```

$ docker context use rootless
$ docker build -t chromium-build .

```

Do not put Dockerfile in the same directory with chromium codebase, otherwise it will take forever to build.

Run the docker container:

```

$ docker run -it \ # interactive shell
    --name chromium-build \ # runtime name of container
    -u root \ # user
    -v /home/zerodaytraining/code/chromium:/chromium \ # mount host's
directories
    -v /home/zerodaytraining/code/depot_tools:/depot_tools \
    chromium-build # name of our image

```

This should drop you to the root shell inside the docker container, in the chromium/src directory exported from the host. Next steps are inside the docker container.

Install build dependencies and run hooks:

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

```

docker# ./build/install-build-deps.sh

# Loop through each directory in /chromium/src/third_party and add
# them as safe directories in Git
docker# for dir in /chromium/src/third_party/*; do
    if [ -d "$dir" ]; then
        git config --global --add safe.directory "$dir"
    fi
Done
docker# git config --global --add safe.directory /chromium/src

docker# gclient runhooks

```

Finally, exit container and commit the changes to the docker database:

```

docker# exit
$ docker container ls -a
$ docker commit <ID from above step> chromium-build:base
# optional cleanup
$ docker image rm chromium-build:latest && docker image prune \
    && docker container prune && docker builder prune

```

Run the container again and start building.

```

$ docker run --rm # cleanup changes after exit
    -it \ # interactive shell
    --name chromium-build \ # runtime name of container
    -u root \
    -v /home/zerodaytraining/code/chromium:/chromium \ # mount host's
directories
    -v /home/zerodaytraining/code/depot_tools:/depot_tools \
    chromium-build:base # name of our updated image

```

Note, building as root user inside the container will produce binaries owned by the current user on the host. On the other hand, using a non-root user as Google recommends causes a chain of permission issues due to the way dockerd handles mounted directories, so I don't recommend it.

Build

Update your checkout:

```

$ git rebase-update
$ gclient sync

```

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

Generate the build config:

```
docker# gn gen out/Default
```

The above command generates a default dev-build config (“release + asserts”) that will produce object files in out/Default directory, where the last name is arbitrary, but `out` part should remain. You only need to run it once for each config/directory.

The build configuration can be modified by calling `gn args`:

```
# show the list of available options with current setting
gn args out/Default --list
# open config in the editor
gn args out/Default
-or-
# pass interesting arguments directly to the configurator
gn gen out/Default --args="....."
```

The actual configuration will be stored in the `out/<yourbuildname>` directory as args.gn file.

Build Chromium:

```
docker# autoninja -C out/Default chrome
```

Building with full symbols takes hours. You can increase the number of cores and RAM in the VM settings to make it faster. Also, refer to the “Faster builds” section in the official build documentation.

After the build has completed, the chrome binary will be in the `out/Default/` directory. You can exit the container and launch/debug it on the host as usual.

Troubleshooting

Building and running older versions of Chrome for bug testing can be tricky. The command `gclient sync` pulls code dependencies from google’s servers and if they were removed, it fails to sync and won’t build. This happens all the time, even with “not so old” versions of chrome from 5 years ago. Likewise, binary dependencies of old chrome builds may cause compatibility issues on newer systems.

SPOILER ALERT! Solution for module 3.4 exercise below.

Especially for our case study, steps to build and run chrome dev+asan are as follows:

1. Checkout the given commit.
2. Checkout depot_tools to around the same date as the commit.

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

3. Comment out dependency on tools_traffic_annotation_linux in DEPS file if `gclient sync` command fails to find it.
4. I had to install common Python 2.7 dependencies on the system where Python 3.8 is the default, such as http.server.

```
$ sudo apt install python-pip && pip2 install <module>
```
5. AddressSanitizer build will randomly crash trying to execute a zeroed code page due to a [known bug related to high-bit ASLR](#) Linux kernel update. Workaround is to start a no-ASLR shell and run the build from there.

```
$ setarch `uname -m` -R $SHELL
```
6. AddressSanitizer build will still randomly crash with global-buffer-overflow reports on static strings, ignore.
7. Run the builds with --no-sandbox command line flag. (Error 159).
8. Cleanup chrome local user directory if it reports a broken sqlite database.

```
$ rm ~/.config/chromium -r
- or -
$ ./chrome --user-data-dir=newdirectory ...args...
```

Safari & WebKit

Safari is Apple's proprietary browser, based on the open source WebKit. WebKit implements the majority of the core browser attack surfaces in Safari, such as the JavaScript engine and the DOM, so WebKit on Linux can be effectively the research target when searching for security vulnerabilities in Safari.

As WebKit is maintained and primarily used by Apple, the "native" OS platform to build it would be Mac OS. On Linux, WebKitGTK port can be built, which is mostly the same core code with the same bugs. Some differences of importance between WebKit and WebKitGTK will be in the graphics layers about OS integration. Building WebKit on Mac can be more complicated than on Linux, I will mention some tips in the lectures. This instruction is to build on Linux.

There are two options to grab WebKitGTK code:

1. Download release tarball from <https://webkitgtk.org/releases/>.
2. Checkout git repo - my choice.

Check out the code:

```
git clone https://github.com/WebKit/WebKit.git WebKit
```

There is only one branch `main` and many tags, inspecting them with `git tag --list` shows a bunch of Safari branches, as well as webkit-gtk and wpe branches.

Install dependencies (this will download 10+ GB):

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

```
sudo apt-get install flatpak
Tools/Scripts/update-webkitgtk-libs
```

Build the GTK port of WebKit - dev configuration:

```
Tools/Scripts/build-webkit --gtk
```

Build debug configuration with symbols:

```
Tools/Scripts/build-webkit --gtk --debug
--cmakeargs="-DDEVELOPER_MODE_FATAL_WARNINGS=OFF"
```

Build with Address Sanitizer (note: may require tuning - build is unstable):

```
Tools/Scripts/build-webkit --gtk --debug
--cmakeargs="-DENABLE_SANITIZERS=address"
```

Run the build (MiniBrowser is the test GUI for WebKit):

```
Tools/Scripts/run-minibrowser --gtk --release|--debug
```

References

- Building Firefox On Linux https://firefox-source-docs.mozilla.org/setup/linux_build.html
- Building and testing SpiderMonkey <https://firefox-source-docs.mozilla.org/js/build.html>
- Configuring Build Options
https://firefox-source-docs.mozilla.org/setup/configuring_build_options.html
- General debugging tips
https://firefox-source-docs.mozilla.org/contributing/debugging/debugging_firefox_with_gdb.htm
- Debugging tips for SpiderMonkey
https://firefox-source-docs.mozilla.org/js/hacking_tips.html
- Build Chromium on Linux
[https://chromium.googlesource.com/chromium/src/+main/docs/linux/build_instructions.md](https://chromium.googlesource.com/chromium/src/+/main/docs/linux/build_instructions.md)
- Docker rootless mode <https://docs.docker.com/engine/security/rootless/>
- Building WebKitGTK <https://trac.webkit.org/wiki/BuildingGtk>

Troubleshooting

This section deals with common scenarios of build issues with open source projects.

Common mistake in this scenario is viewing build issues as isolated breakages and chasing to fix them, which produces more issues and eventually may ruin the build. Rather, in mature codebases build issues are more likely to be systematic, meaning that something would be wrong at large with the overall setup.

OOM Workarounds

If your system is short on operating memory, a large project build process can cause an OOM (Out Of Memory) condition. Modern Linux kernel has a guard that will start killing the offending and random processes in this situation. That is frustrating, and in some situations a critical issue, like CodeQL builds which don't support resumes.

Disabling kernel OOM guards won't help, as it doesn't address the underlying root cause. The obvious solution is to extend physical memory, if that isn't an option, workarounds below.

1. Reduce parallelism in the build config.

Modern build systems parallelize, i.e. they will try to run multiple compilation and linking processes simultaneously whenever multiple CPU cores are available. If the box has less RAM than it should based on the core count, and the build takes all the cores (default), OOM can easily happen with certain build configurations. See the CPU & RAM section for more details.

Majority of modern build systems on Linux are based on either GNU make or ninja, that both support the same argument convention “-j N” for managing CPU level parallelisation. For example:

```
$ make -j1 // turn off parallelization
$ make -j$(nproc) // use all the CPU cores available
$ ninja -j2 // use two cores
```

In large projects makeflags aren't guaranteed to be exposed to the top level build scripts, so you'll have to look under the hood of the build system to find out how to change it.

2. Enlarge system swap file.

You can offset RAM requirements with some extra disk space. Forcing the system to swap more will obviously cost performance, but at least the build can finish. Set the swapfile size to 4G or ¼-th of the total RAM size, whichever is larger. On Linux as root:

```
root# swapoff /swapfile
root# rm /swapfile
root# fallocate -l 4G /swapfile
root# chmod 600 /swapfile
root# mkswap /swapfile
root# swapon /swapfile
```

Package Dependency Issues

Oftentimes the project's build instructions will be incomplete, incorrect or outdated, and you'll run into various issues. Package dependency issues are most common, project dependency issues are second most common. This section deals with the former.

1. Use a clean installation of the build platform OS.

System packages that you install as part of security updates or system upgrades can be incompatible with the build, if the build requires a different version of the package. A lesser version requirement may be non downgradable, while a higher version package required may be not available for upgrade from the package manager.

Solid modern projects are tested on a clean-install OS version which is explicitly listed as supported in the build instructions. If this solution doesn't help, you may need to fix packages manually.

2. Fixing packages.

To fix package dependency issues, you may need to downgrade or upgrade packages manually.

To downgrade a package, remove it manually using the package manager command line and install the specific version required by the build. This may need to be repeated down the chain of dependency.

To upgrade the package which isn't available in the package manager, you can try to install it manually by downloading it from the internet.

If you must start messing with packages, make sure to backup the system, as it can break it.

3. Remove/ignore unnecessary build targets.

Package dependency issues can come from some unnecessary build target, like project documentation. Review build targets and keep just what you need.

Also, you can try to pass the `--keep-going` or `-k` flag to the build system and hope that despite any errors it will build what is needed.

Project Dependency Issues

Most C++ level build errors in public open source projects come from internal project dependency issues: such as, using a wrong version of C++ lib headers, a wrong version of C++ language passed to compiler, or messed up dependencies with 3rd party libraries.

These kinds of issues aren't common when building major browser projects on Linux. They are however common on Mac OS, regardless of project maturity and OS version. They are also common when building less popular codebases on Linux.

Project internal dependency issues can be subtle, with few boilerplate solutions. In some cases you may need to apply weird hacks of build scripts or project files.

Common root causes:

1. Version mismatch of dependency codebases.

Large projects use 3rd party codebases, such as open source libraries, that will be checked out under the main directory, but managed as separate code trees with their own `.git` source. Meaning, dependent codebases must be somehow synchronised with the main code tree as commits roll.

Modern projects solve this problem internally; for example, Google's `depot_tools` automates syncing of 3rd party project dependencies (which is why you'll call `gsync` after each checkout).

So, make sure that the checkout commits of dependency codebases are correct per the main tree commit that you are building.

2. Version mismatch of C++ system headers.

A rare issue, doesn't usually happen on popular codebases. It manifests as a failure to find a member of some C++-internal class. Solution usually would be to update system C++ library packages. Another helpful idea is to check header directories that the build system includes under the hood, they may be wrong.

3. Incompatible system libraries.

Also a rare issue, it manifests as a failure to link object files against system libraries. Most projects will verify system dependencies before starting to build. If this issue happens, it usually

© 2025 Zero Day Engineering LLC. All rights reserved.

Confidentiality: Exclusive to attendees of Browser Exploit Design training.

Last updated: 05.06.2025

means that the build is broken in some systematic way, and advisable to set up from scratch. Another idea is to check paths passed to the linker, they may be incorrect.

VirtualBox x Ubuntu Issues

If the VM GUI freezes after inputting Ubuntu login password, switch to another *tty* and back with Ctrl + Alt + F1/F2. This should load the desktop. You can use the soft keyboard accessible from the VM menu for that.